

Diccionario de Palabras Clave — Semana 2

Bootcamp: De Cero a Data Engineer - Databricks SQL

Bloque temático: ANALIZAR | Clase Grupal

Instructor: Luciano Argolo

Este diccionario recopila todos los términos técnicos discutidos en la clase grupal de la Semana 2. Cada definición está escrita para alguien que nunca trabajó con datos ni programación. Usalo como referencia rápida durante los ejercicios y para preparar entrevistas.

Git y Control de Versiones

Comandos, ramas, commits y flujos de trabajo colaborativo.

SQL y Consultas

CTEs, subqueries, filtros, joins y funciones avanzadas.

Window Functions

Cálculos sobre ventanas de filas sin colapsar resultados.

Arquitectura Medallion

Capas Bronze, Silver y Gold para organizar datos.

Databricks y Herramientas

Plataforma, catálogo, clusters, Spark y más.

Estadística y Calidad

EDA, outliers, percentiles, gobernanza y compliance.

Git y Control de Versiones

Git	Programa que registra cada cambio en el código: quién lo hizo, cuándo y qué modificó. Se opera con comandos como <code>git status</code> , <code>git add</code> , <code>git commit</code> , <code>git push</code> .
GitHub	Plataforma web donde se guardan repositorios de código. Permite colaborar en equipo y sirve como portfolio profesional para entrevistas.
Repositorio (repo)	Espacio que almacena código con todo su historial de cambios. Puede ser local (tu PC) o remoto (GitHub).
Branch (rama)	Copia paralela del código para trabajar sin afectar la versión principal. Las ramas se crean, se trabajan y después se unen al tronco.
Commit	"Foto" de los cambios que hiciste, con un mensaje que explica qué modificaste. Queda en el historial para siempre.
Push	Subir tus commits locales a GitHub. Hasta que no hacés push, los cambios solo existen en tu computadora.
Pull	Descargar los cambios más recientes desde GitHub a tu copia local. Para mantenerse actualizado con el equipo.
Pull Request (PR)	Solicitud para que alguien revise tus cambios antes de integrarlos a la rama principal. El líder revisa y aprueba.
Merge	Combinar los cambios de una rama con otra. Puede generar conflictos si dos personas tocaron lo mismo.
GitFlow	Patrón de trabajo con ramas: <code>main</code> (producción), <code>develop</code> (desarrollo), <code>features</code> (funcionalidades) y <code>hotfix</code> (arreglos urgentes).
Hotfix	Arreglo urgente directo a producción sin pasar por todo el flujo. Se usa cuando algo se rompió y hay que arreglarlo ya.
Feature	Rama para trabajar en algo específico. Se crea, se trabaja y al terminar se integra al flujo principal.
Git Bash	Terminal que viene con Git en Windows. Permite ejecutar comandos de Git y Linux. Hay que instalarla aparte.
PAT	Personal Access Token: clave de GitHub para conectar Databricks con tu repositorio. Funciona como contraseña especial.
SourceTree / GitKraken	Herramientas con interfaz gráfica para Git. Permiten ver branches con gráficos y colores en vez de usar la terminal.
WSL	Windows Subsystem for Linux: correr Linux dentro de Windows. Útil para Docker, librerías de Python y repos que requieren Linux.

SQL y Consultas

CTE	Common Table Expression: query temporal con nombre usando WITH . Le ponés un apodo a una query para reutilizarla. Se encadenan varias para construir queries complejas de forma legible.
Subquery	Query dentro de otra query. Hace lo mismo que una CTE pero el código queda anidado y difícil de leer. Las CTEs son la versión limpia.
GROUP BY	Agrupar filas con el mismo valor para hacer cálculos (suma, promedio). El resultado tiene menos filas: se pierde granularidad a cambio de datos agregados.
HAVING	Filtra resultados después del GROUP BY . A diferencia de WHERE que filtra antes de agrupar, HAVING filtra sobre los grupos ya formados.
WHERE	Filtra filas antes de cualquier agrupación. Es de las primeras cosas que se evalúa en una query SQL.
CASE WHEN	Instrucción para crear condiciones: "si pasa esto, entonces hacé aquello". Similar a if/else en programación.
CAST	Convierte un dato de un tipo a otro (texto a número, texto a fecha). Se usa al pasar de bronze (String) a silver (tipos correctos).
CROSS JOIN	Combina cada fila de una tabla con todas las filas de otra. Se usa para comparar cada registro contra un valor global como el promedio.
Self-join	JOIN de una tabla consigo misma. Más complejo que Window Functions pero a veces necesario.
Store Procedure	Función guardada en la base de datos que encapsula queries SQL reutilizables. Databricks SQL no los soporta; es más de SQL Server.
QUALIFY	Filtra resultados de Window Functions, análogo a HAVING para GROUP BY . Databricks lo soporta; MySQL y PostgreSQL estándar, no.

Funciones de Ventana (Window Functions)

¿Qué es una Window Function?

Función SQL que hace cálculos sobre un grupo de filas **sin colapsar el resultado**. Mantiene todas las filas y agrega columnas con promedios, rankings o comparaciones.

Diferencia clave

GROUP BY → reduce filas (pierde granularidad)

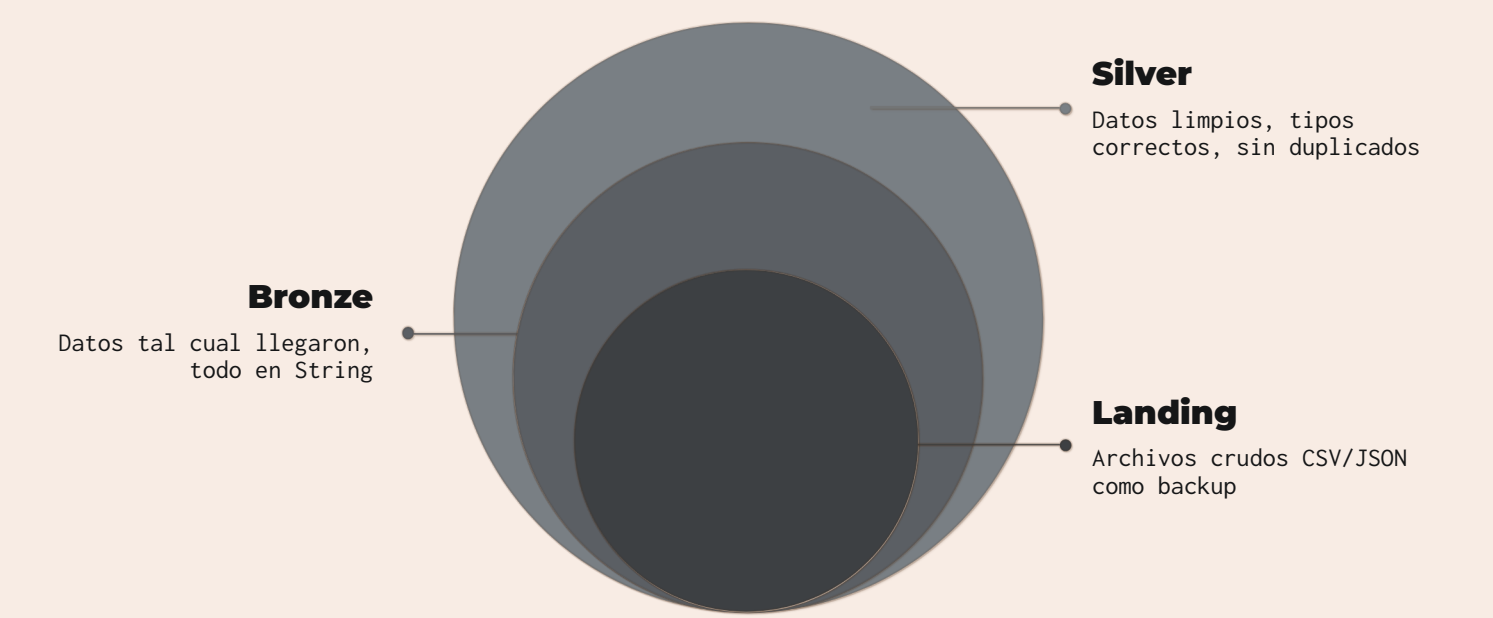
PARTITION BY → mantiene todas las filas

Sintaxis general

```
FUNCION() OVER (
  PARTITION BY columna
  ORDER BY columna
  ROWS BETWEEN ...
)
```

Window Function	Función SQL que hace cálculos sobre un grupo de filas sin colapsar el resultado. Mantiene todas las filas y agrega columnas con promedios, rankings o comparaciones.
PARTITION BY	Dentro de una Window Function, define cómo segmentar los datos. Es el equivalente al GROUP BY pero sin perder filas.
ROW_NUMBER	Asigna un número secuencial único a cada fila, sin importar empates (1, 2, 3, 4...). Aliado clave para eliminar duplicados.
RANK	Asigna ranking manejando empates: si dos empatan en 2, la siguiente es 4 (salta posiciones).
DENSE_RANK	Como RANK pero sin saltar posiciones: si dos empatan en 2, la siguiente es 3.
LAG	Accede al valor de la fila anterior. Sirve para comparar con el período pasado (ventas este mes vs. mes anterior) sin self-join.
LEAD	Accede al valor de la fila siguiente. Lo opuesto a LAG.
AVG() OVER / SUM() OVER	Calcula promedio o suma como Window Function: agrega la columna al lado de cada fila sin colapsar datos.
ROWS BETWEEN	Define el rango de filas que la Window Function considera. UNBOUNDED PRECEDING = desde la primera fila. CURRENT ROW = hasta la fila actual.

Arquitectura de Datos (Medallion)



Medallion	Patrón para organizar datos en capas: Bronze → Silver → Gold. Cada capa agrega más calidad y estructura. Opcionalmente se agrega Landing antes de Bronze.
Landing	Capa previa a Bronze donde se guardan archivos crudos (CSV, JSON, HTML) como backup. Si todo se borra, landing permite reprocesar desde cero.
Bronze	Primera capa: datos tal cual llegaron, todo en String. "Arqueología del dato": se preserva sin tocar para auditoría.
Silver	Segunda capa: datos limpios, con tipos correctos, sin duplicados, estandarizados. Se aplican reglas de negocio básicas.
Gold	Última capa: datos modelados (estrella, snowflake) y optimizados para consumo por áreas de negocio.
Managed table	Tabla que guarda datos internamente. Si se borra la tabla, se pierden los datos. Tipo por defecto en Databricks. Ideal para Silver/Gold.
External table	Tabla que apunta a datos en otra ubicación. Si se borra la tabla, los datos siguen existiendo. Ideal para Bronze leyendo de Landing.
Pipeline	Secuencia automatizada de pasos que transforman datos desde una fuente hasta un destino. Como una línea de producción de fábrica.
Batch	Procesar datos en tandas (cada X minutos/horas). Más eficiente y barato que real-time.
Real-time	Procesar datos al instante. Para trading, alertas de fraude. Costoso y complejo.
Micro batch	Procesamiento por lotes en intervalos muy cortos (segundos). Intermedio entre batch y real-time.
OLTP	Procesamiento transaccional: operaciones rápidas (compras, transferencias). MySQL, PostgreSQL están optimizados para esto.
OLAP	Procesamiento analítico: consultas complejas sobre grandes volúmenes para obtener insights. Es lo que hacemos en el pipeline.

Databricks y Herramientas

Databricks	Plataforma en la nube que combina procesamiento de datos, SQL, Python y más. Tiene notebooks, clusters, catálogo de datos y automatización. Ofrece edición gratuita.
Unity Catalog	Sistema de gobernanza de Databricks. Gestiona catálogos, esquemas, tablas y permisos. Permite accesos granulares y masking de datos sensibles.
Catálogo (Catalog)	Nivel más alto de organización en Unity Catalog. Catálogo = biblioteca, Esquema = estante, Tabla = libro.
Esquema (Schema)	Nivel de organización que agrupa tablas relacionadas dentro de un catálogo.
Volumen (Volume)	Espacio en Unity Catalog para almacenar archivos (CSV, JSON). Como una carpeta dentro del catálogo para archivos crudos.
Cluster	Grupo de computadoras (nodos) que trabajan juntas para procesar datos. Tarda 30-60 segundos en levantar.
Serverless	Modelo donde no gestionás servidores: la plataforma se encarga automáticamente. En Databricks, el cluster se levanta y apaga según demanda.
Spark	Motor de procesamiento distribuido. Es el "cerebro" que Databricks usa para ejecutar queries sobre grandes volúmenes de datos.
Delta Lake	Tecnología de almacenamiento de Databricks: transacciones, viaje en el tiempo, optimización de consultas sobre archivos.
Lakehouse	Arquitectura que combina data lake (flexible, barato) + data warehouse (estructura, gobernanza). Databricks usa esta arquitectura.
Scheduler	Funcionalidad para programar ejecución automática de pipelines. "Ejecutar cada 15 minutos" o "todos los días a las 8am".
Metastore	Almacén de metadatos que sabe qué archivos, tablas y datos están disponibles, dónde están y quién puede acceder.
Docker	Herramienta para crear "contenedores": ambientes aislados con todo lo necesario para que un programa funcione en cualquier computadora.
Kafka	Plataforma para flujos de datos en tiempo real. Sistema de mensajería donde datos se envían/reciben como eventos continuos.
Airflow	Herramienta para programar y automatizar pipelines. Ejecuta tareas en orden/horario y avisa si algo falla.

Estadística y Calidad de Datos

EDA	Exploratory Data Analysis: proceso de explorar datos antes de usarlos: estructura, nulos, outliers, estadísticas. Paso obligatorio antes de construir un pipeline.
Outlier	Dato que se sale del patrón normal (ej: viaje de 0.01 millas, alquiler de 1 dólar). Se detecta con EDA y se maneja con percentiles.
Promedio (media)	Suma de valores dividida por cantidad. Sensible a outliers: un valor extremo puede distorsionar todo el resultado.
Mediana	Valor del medio al ordenar todos los datos. Robusta ante outliers. El percentil 50.
Moda	Valor que más se repite. Viene de "moda" = lo más frecuente/popular.
Media recortada	Promedio que descarta extremos (ej: 5% inferior y 5% superior). Se queda con el 90% central para evitar distorsión por outliers.
Percentil	Posición relativa de un valor. Percentil 50 = mediana. Percentil 95 = el 95% de datos está por debajo. Se usa para detectar outliers.
Sesgo (bias)	Datos inclinados por un factor que distorsiona. Ejemplo: el promedio de ingresos en Medina (Washington) es altísimo por Bill Gates y Bezos.
Desviación estándar	Qué tan dispersos están los datos respecto al promedio. El 96% de datos suele estar dentro de 2 desviaciones estándar.
Curva de Gauss	Forma de campana cuando datos se distribuyen normalmente: mayoría en el centro, pocos en extremos.
Intervalo de confianza	Rango donde esperamos el valor real con cierta probabilidad. En ciencia de datos, 5% es el umbral estándar.
Nulo (NULL)	Ausencia de valor en un campo. No es cero ni vacío. Detectar nulos es lo primero que se hace en un EDA.
Gobernanza de datos	Reglas sobre quién accede a qué datos, cómo se protegen y cómo se asegura calidad. Unity Catalog lo implementa en Databricks.
PII	Datos que identifican a una persona (nombre, DNI, email). Deben protegerse por compliance (GDPR, CCPA).
Masking	Ocultar datos sensibles: la persona ve la tabla pero no el contenido de columnas protegidas (ej: DNI enmascarado).
Compliance	Cumplir normas como GDPR (Europa) o CCPA (California). Proteger datos personales, permitir borrado en 30 días, etc.
Linaje de datos	Rastrear de dónde vino un dato, qué transformaciones pasó y a dónde fue. Unity Catalog lo ofrece automáticamente.
KPI	Métrica clave que mide rendimiento (promedio de ventas, % de nulos). Sirve para tomar decisiones basadas en datos.